



stkADX v2 Security Review

heyAura

Table of Contents

01	Review parameters	↗
02	Executive summary	↗
03	Findings	↗
04	Informational observations	↗
05	Appendix: verification and limitations	↗



Review parameters

PREPARED BY	GOODMORNING
Prepared for	heyAura, <code>adex-protocol-eth</code>
Date	2026-08-30
Branch	<code>feature/aura-staking</code>
Head reviewed	<code>a7c82a9</code>
Scope	<code>contracts/StakingPool.sol</code> , <code>contracts/StakingPoolMigratorGovernance.sol</code> , <code>contracts/SupplyController.sol</code> , and the interfaces required by those contracts

Executive summary

goodmorning reviewed the stkADX v2 staking, migration, and incentive contracts at commit `a7c82a9`.

The report contains twelve findings. The team resolved nine findings before this report. Three findings remained open at the reviewed commit.

The report also records nine informational observations. These observations cover deployment requirements, trust assumptions, accounting, build configuration, and gas use.

STATUS	COUNT	ENTRIES	MEANING
Resolved	9	A-01 – A-09	The reviewed commit contains the stated code change
Open	3	N-01 – N-03	The reviewed commit does not contain a correction
Informational observations	9	N-04 – N-05, N-07 – N-11, N-14, N-18	Context or defensive improvements without a formal severity

Terms used in this report

TERM	MEANING
Base token	ADX held by the pool and returned when a staker exits
Share	An internal accounting unit that represents part of the pool
Share value	The base-token value represented by <code>1e18</code> shares
Token amount	The external stkADX amount used by <code>balanceOf</code> , transfers, approvals, and events
Allowance	The token amount that an owner authorizes a spender to transfer
Commitment	An unbonding record containing shares, an unlock time, and the maximum base-token payout
Unbonding	The waiting period between <code>unstake</code> and <code>withdraw</code>
Rage leave	An immediate exit that pays the configured fraction of the position
Mintable incentive	ADX rewards reported by <code>SupplyController</code> but not yet minted into the pool
Permit domain	The EIP-712 value that binds a permit signature to a chain and contract
Minting headroom	The amount that <code>SupplyController</code> can still mint before reaching its cap
Promille	A part per thousand; 1000 promille equals 100%

Findings

Each entry separates the observed code, required conditions, consequence, action, and verification.

Unless stated otherwise, verification used source inspection. The review did not execute regression tests for the individual findings.

Findings summary

ID	SEVERITY	FINDING	STATUS	RESOLUTION
A-01	High	<code>transferFrom</code> deducted shares from a token-denominated allowance	Resolved	<code>c24fffb</code>
A-02	Medium	<code>transferFrom</code> checked the spender's commitment instead of the owner's commitment	Resolved	<code>8f72ed1</code>
A-03	Medium	<code>Transfer</code> events reported shares instead of token amounts	Resolved	<code>8f72ed1</code> , <code>495ee1d</code>
A-04	Medium	Rage leave could leave a commitment larger than the remaining share balance	Resolved	<code>be4939d</code>
A-05	Medium	A dust transfer could emit an event without moving shares	Resolved	<code>6e1fa68</code>
A-06	Medium	ERC-20 operations used the ADX-specific interface	Resolved	<code>7ae8466</code> , <code>59b26f1</code>
A-07	Low	<code>transferFrom</code> reduced unlimited allowances	Resolved	<code>cb867ee</code>
A-08	Low	<code>transferFrom</code> allowed the pool as the recipient	Resolved	<code>119c3f0</code>
A-09	Low	Share rounding caused excess allowance deduction	Resolved	<code>c534560</code>
N-01	Medium	Third-party deposits can block a user from lowering their unbond time	Open	Pending
N-02	Low	A fixed <code>permit</code> domain can allow replay after a chain split	Open	Pending
N-03	Low	<code>setIncentive</code> discards accrued rewards	Open	Pending

A-01 · High · `transferFrom` deducted shares from a token-denominated allowance

Status Resolved · Team response Implemented before final report · Resolved in `c24fffb`

```
-    allowed[from][msg.sender] = allowed[from][msg.sender] - shareAmount;  
+    allowed[from][msg.sender] = allowed[from][msg.sender] - amount;
```

Fact. The contract stores allowances in token units after `approve` or `permit`.

The old `transferFrom` function converted the requested amount to shares. It then deducted that share count from the allowance.

Preconditions. The share value must exceed `1e18`. The owner must also hold enough value for repeated transfers.

Transaction sequence.

The share value reaches `2e18`.

An owner authorizes a spender to transfer 100 token units.

The spender requests 100 token units, which converts to 50 shares.

The old code deducts 50 allowance units.

The spender repeats the transfer while the owner has sufficient shares.

Consequence. The spender can transfer more token value than the owner authorized. In this example, a 100-unit allowance permits 200 units.

Resolution. Commit `c24fffb` changed the deduction to token units. Commit `c534560` later matched the deduction to the token value transferred.

Verification. The reviewed code deducts $(\text{shareAmount} * \text{_shareValue}) / 1e18$. This expression uses token units rather than share units.

A-02 · Medium · `transferFrom` checked the spender's commitment instead of the owner's commitment

Status Resolved · Team response Implemented before final report · Resolved in `8f72ed1`

```
- require(commitments[msg.sender].shareAmount == 0, "unstaking in progress");  
+ require(commitments[from].shareAmount == 0, "unstaking in progress");
```

Fact. An open commitment should stop its owner from transferring the committed position.

The old `transferFrom` function checked `msg.sender`. In this function, `msg.sender` is the spender. The `from` address is the owner.

Preconditions. The owner must authorize a spender. Either the owner or the spender must also have an open commitment.

Transaction sequence.

The owner opens an unbonding commitment.

A spender without a commitment calls `transferFrom`.

The old check reads the spender's empty commitment.

The transfer moves the owner's shares during unbonding.

A spender with a commitment could not transfer for an owner who had no commitment.

Consequence. The first case bypassed the owner's transfer lock. The second case blocked an unrelated transfer.

Resolution. Commit `8f72ed1` changed the check from `commitments[msg.sender]` to `commitments[from]`.

Verification. The reviewed function checks the commitment that belongs to `from`.

A-03 · Medium · **Transfer** events reported shares instead of token amounts

Status Resolved · Team response Implemented before final report · Resolved in [8f72ed1](#) , [495ee1d](#)

```
-      emit Transfer(from, to, shareAmount);  
+      emit Transfer(from, to, amount);
```

Fact. The ERC-20 interface reports balances and allowances in token units. The old transfer events reported the internal share count.

Preconditions. A transfer must occur while the share value differs from [1e18](#) . An external system must also consume the event value.

Consequence. The event value differs from the amount requested through the ERC-20 interface. Event-based accounting can therefore disagree with [balanceOf](#) .

Resolution. Commit [8f72ed1](#) corrected [transferFrom](#) . Commit [495ee1d](#) applied the same change to [transfer](#) .

Verification. Both functions emit the requested token amount in [a7c82a9](#) .

A-04 · Medium · Rage leave could leave a commitment larger than the remaining share balance

Status Resolved · Team response Implemented before final report · Resolved in [be4939d](#)

```
+     if (commitments[msg.sender].shareAmount > shares[msg.sender]) {  
+         commitments[msg.sender] = UnbondCommitment({ unlocksAt: 0, tokensToReceive: 0,  
+             shareAmount: 0 });  
+     }
```

Fact. `unstake` records a commitment. It leaves the committed shares in the user's balance. The old `rageLeave` function did not update that commitment.

Preconditions. A user must open a commitment and then rage-leave enough shares to make the remaining balance smaller than the commitment.

Transaction sequence.

A user commits 100 shares through `unstake`.

The user rage-leaves those 100 shares before calling `withdraw`.

The share balance becomes zero, while the commitment still records 100 shares.

`withdraw` tries to burn more shares than the user owns and reverts.

`transfer` and `transferFrom` remain blocked while the commitment exists.

Consequence. The user cannot withdraw or transfer through the normal paths. The user must first restore enough shares to satisfy the commitment.

Resolution. Commit [be4939d](#) clears the commitment when its share amount exceeds the remaining share balance.

Verification. The reviewed `rageLeave` function performs this check after burning shares.

A-05 · Medium · A dust transfer could emit an event without moving shares

Status Resolved · Team response Implemented before final report · Resolved in [6e1fa68](#)

```
+ require(shareAmount > 0, "trying to transfer zero shares");
```

Fact. The conversion `amount * 1e18 / shareValue` rounds down. A positive token amount can therefore convert to zero shares.

The old functions accepted that conversion. They moved zero shares. They also emitted a nonzero `Transfer` event.

The old `transferFrom` function also reduced the allowance.

Preconditions. The requested amount must be positive and smaller than the token value of one share unit.

Consequence. Pool balances do not change. An event-only integration can record a transfer that did not occur.

The false event contains less value than one share unit. Each repeated call requires another transaction. Each call also costs gas.

Resolution. Commit [6e1fa68](#) rejects transfers when the converted share amount equals zero.

Verification. Both transfer functions contain the new guard in [a7c82a9](#).

A-06 · Medium · ERC-20 operations used the ADX-specific interface

Status Resolved · Team response Implemented before final report · Resolved in [7ae8466](#), [59b26f1](#)

```
- require(IADXTOKEN(baseToken).transferFrom(msg.sender, address(this), amount));  
+ require(IERC20(baseToken).transferFrom(msg.sender, address(this), amount));
```

Fact. Balance and transfer operations require the ERC-20 interface. Earlier revisions called those operations through `IADXTOKEN`.

Preconditions. No transaction triggers this issue. The interface choice affects source clarity and future integration work.

Consequence. The interface choice obscured the pool's actual dependencies. Both interface types call the same functions at runtime.

Resolution. Commits [7ae8466](#) and [59b26f1](#) changed standard token operations to `IERC20`.

The pool remains ADX-specific because its incentive calls require `IADXTOKEN`.

Verification. The reviewed code uses `IERC20` for balances and transfers. It uses `IADXTOKEN` when calling `supplyController`.

A-07 · Low · `transferFrom` reduced unlimited allowances

Status Resolved · Team response Implemented before final report · Resolved in `cb867ee`

```
-    allowed[from][msg.sender] = allowed[from][msg.sender] - amount;
+    uint256 prevAllowance = allowed[from][msg.sender];
+    if (prevAllowance < type(uint256).max) allowed[from][msg.sender] = prevAllowance - amount;
```

Fact. ERC-20 integrations commonly use `type(uint256).max` to represent an unlimited allowance.

The old function reduced this value after each transfer.

Preconditions. An owner must set the allowance to `type(uint256).max`. The spender must then call `transferFrom`.

Consequence. The allowance becomes finite after the first transfer. Each transfer also performs an unnecessary storage write.

Resolution. Commit `cb867ee` leaves the maximum allowance unchanged.

Verification. The reviewed function updates allowances only when `prevAllowance` is smaller than `type(uint256).max`.

A-08 · Low · `transferFrom` allowed the pool as the recipient

Status Resolved · Team response Implemented before final report · Resolved in `119c3f0`

```
+    require(to != address(this), "cannot send to contract");
```

Fact. The old `transferFrom` function allowed `to` to equal the pool address.

Preconditions. An owner or approved spender must transfer `stkADX` to the pool address.

Consequence. The contract credits shares to its own address. The pool provides no function that spends shares owned by the pool address.

Those shares remain in `totalShares`. The sender cannot access them through the normal staking interface.

Resolution. Commit `119c3f0` makes both transfer functions reject the pool address before changing balances.

Verification. The reviewed functions start with `require(to != address(this))`.

A-09 · Low · Share rounding caused excess allowance deduction

Status Resolved · Team response Implemented before final report · Resolved in [c534560](#)

```
-    if (prevAllowance < type(uint256).max) allowed[from][msg.sender] = prevAllowance - amount;  
+    if (prevAllowance < type(uint256).max) allowed[from][msg.sender] = prevAllowance -  
    (shareAmount * _shareValue) / 1e18;
```

Fact. Token-to-share conversion rounds down. The resulting shares can represent slightly less value than the requested token amount.

The old function deducted the full requested amount from the allowance.

Preconditions. The conversion must leave a rounding remainder. The owner must use a finite allowance.

Consequence. The allowance falls by more than the token value removed from the owner's balance. The difference remains below one share unit's token value.

Resolution. Commit [c534560](#) deducts the token value represented by the transferred shares.

Verification. The reviewed function uses the same cached share value for conversion and allowance deduction.

The following findings remained open at commit [a7c82a9](#).

N-01 · Medium · Third-party deposits can block a user from lowering their unbond time

Status Open · Team response Pending · Resolution Not submitted

```
function enterWithUnbondTime(uint amount, uint time) external {
    require(time >= individualTimeToUnbond[msg.sender] || shares[msg.sender] == 0,
        "you cannot reduce individual time to unbond unless you have zero stake");
```

Fact. A user can lower their individual unbond time only when their share balance equals zero.

`enterTo` lets any caller deposit for any recipient. The recipient does not approve the deposit.

Preconditions. The recipient must have a longer individual time and must want to reduce it. An attacker must submit `enterTo` before the reset.

Transaction sequence.

The recipient exits their existing shares.

The attacker calls `enterTo` with enough ADX to mint at least one share.

The recipient calls `enterWithUnbondTime` with a shorter time.

The balance check fails because the recipient owns the new share.

Consequence. The recipient keeps the previous unbond time. The attacker can repeat the deposit after the recipient removes the unwanted share.

Removing each unwanted share costs the recipient gas. This sequence gives the attacker no access to the recipient's existing stake.

Recommendation. Require recipient authorization for third-party deposits, or track voluntary and unsolicited shares separately when applying the reset rule.

Verification. The reviewed `enterTo` path changes `shares[recipient]` without recipient authorization. The branch has no regression test for this sequence.

N-02 · Low · A fixed `permit` domain can allow replay after a chain split

Status Open · Team response Pending · Resolution Not submitted

```
constructor(IADXTOKEN token, address governanceAddr) {  
    ...  
    uint chainId;  
    assembly { chainId := chainid() }  
    DOMAIN_SEPARATOR = keccak256(abi.encode(..., chainId, address(this)));  
}
```

Fact. The constructor computes `DOMAIN_SEPARATOR` once. The `permit` function continues using that stored value after the chain ID changes.

Preconditions. A chain split must change the chain ID. The contract address, owner nonce, signature deadline, and signed permit data must remain usable.

Transaction sequence.

An owner signs a permit before the deadline.

The chain splits before both branches consume the permit nonce.

A spender submits the signature on one branch.

The spender submits the same signature on the other branch while its nonce still matches.

Consequence. Both branches can accept the same permit signature. Each branch creates the signed allowance independently.

Recommendation. Cache the deployment separator and rebuild it when `block.chainid` or `address(this)` differs from the cached values.

Verification. The reviewed constructor stores one separator. The reviewed `permit` function never compares the current chain ID with the deployment chain ID.

N-03 · Low · `setIncentive` discards accrued rewards

Status Open · Team response Pending · Resolution Not submitted

```
function setIncentive(address addr, uint amountPerSecond) external {
    require(governance[msg.sender] >= uint8(GovernanceLevel.All), "NOT_GOVERNANCE");
    require(amountPerSecond < 1e18, "AMOUNT_TOO_LARGE");
    incentiveLastMint[addr] = block.timestamp;
    incentivePerSecond[addr] = amountPerSecond;
}
```

Fact. `mintableIncentive` calculates rewards from the last mint timestamp and the current rate.

`setIncentive` replaces both inputs without minting the accrued amount first.

Preconditions. The address must have a nonzero incentive rate and unminted rewards. Governance must call `setIncentive` before `mintIncentive`.

Transaction sequence.

Rewards accrue under the existing rate.

Governance calls `setIncentive`.

The function sets `incentiveLastMint` to the current timestamp.

The previous accrual no longer appears in `mintableIncentive`.

Consequence. The controller can no longer mint the previous accrual through the incentive function.

At 0.1 ADX per second, a 30-day gap accrues about 259,200 ADX. The reset also lowers the pool's reported supply.

Recommendation. Move reward minting into an internal function. Call it before `setIncentive` changes the timestamp or rate.

Until the code changes, governance should execute `mintIncentive(addr)` before `setIncentive`. An atomic governance batch prevents another action from entering between them.

Verification. The reviewed function resets the timestamp without minting the accrued amount. The branch has no test for a rate change with pending rewards.

Informational observations

These observations record deployment conditions, trust assumptions, accounting behaviour, build configuration, and gas costs.

N-04 · The pinned compiler cannot build the staking contracts

Type Informational

Fact. The staking sources comment out their Solidity pragmas. `truffle-config.js` pins Solidity 0.8.7.

The contracts omit `override` markers that Solidity 0.8.7 requires. Solidity 0.8.8 accepts the same declarations.

Preconditions. A developer must build commit `a7c82a9` with the repository's pinned configuration.

Consequence. The configured build rejects the staking contracts. Developers cannot reproduce deployment bytecode from this commit without changing the configuration.

Recommendation. Add an explicit pragma of at least Solidity 0.8.8 to each staking source. Pin the build configuration to the same compiler version.

Verification. Separate `solc-js` runs produced the results in the appendix.

N-05 · The grace-period parameter uses an absolute timestamp

Type Informational

Fact. The constructor assigns a nonzero `_customGracePeriod` directly to `gracePeriod`. It adds `_customDeadline` to that timestamp as a duration.

The parameter names do not explain this difference. The default branch calculates both values from durations.

Preconditions. A deployer must pass a duration, such as `32 days`, as `_customGracePeriod`.

Consequence. The contract interprets that duration as a timestamp in 1970. The configured migration burn schedule can therefore start immediately.

Recommendation. Accept one time convention for both parameters. Validate the resulting grace timestamp and deadline during construction.

Verification. The reviewed constructor contains both time operations. It has no timing validation.

N-07 · Migration leaves stale v1 unbonding records

Type Informational

Fact. The v1 `transferFrom` function can move shares recorded in a pending v1 unbond. The migration then rage-leaves those shares.

The migration does not clear the v1 commitment or `lockedShares` record.

Preconditions. A user must migrate while a v1 unbond remains pending.

Transaction sequence.

- The user approves the migrator for the v1 position.

- The migrator transfers the v1 stake, including the staged shares.

- The migrator rage-leaves those shares.

- The migrator deposits the received ADX into v2.

- The v1 commitment and locked-share record remain unchanged.

Consequence. The stale records cannot create a second withdrawal because the v1 balance has already decreased.

`scripts/tomPendingToLeave.js` can still count those records as pending value.

Recommendation. Update the reconciliation script to exclude users who migrated a staged v1 unbond. The migration contract requires no change for double-spend prevention.

Verification. The review compared the migrator with the deployed v1 pool source. The v1 transfer path moves the staged shares.

N-08 · Future pools require atomic seeding

Type Informational · **Applies to** New deployments

Fact. `innerEnter` can mint zero shares because integer division rounds down. The function still transfers the depositor's ADX after that calculation.

Preconditions. A pool must start without shares. An attacker must create a small initial share supply and then donate ADX directly.

Transaction sequence.

The attacker makes the first small deposit.

The attacker transfers additional ADX directly to the pool.

The donation raises the value of each share.

A later deposit rounds `newShares` to zero.

The pool receives the later depositor's ADX without issuing shares.

Consequence. The later depositor receives no pool position for that deposit.

The existing deployment received its initial seed before this review. This observation applies to future pool deployments.

Recommendation. Seed each new pool atomically with deployment. Reject any deposit for which `newShares` equals zero.

Verification. Static review confirms that `innerEnter` lacks a nonzero-share check. This review did not reconstruct the current pool's full share history.

N-09 · The caller supplies both pool addresses

Type Informational · **Assumption** Callers use trusted pool contracts

Fact. `migrate` accepts `oldPool` and `newPool` as call parameters. The contract does not compare them with an allowlist.

The migrator calls both addresses. It also approves the supplied new pool to spend ADX.

Preconditions. A caller must supply an unexpected or incompatible pool address.

Consequence. An unexpected address receives the migrator's external call or token approval. A malicious pool can control the called behaviour.

Recommendation. Store the approved pool addresses as immutable settings, or validate them against a governance-managed allowlist.

Also verify the expected pool version before migration. The v1 and v2 contracts use different share and token relationships.

Verification. The reviewed function lets the caller select both addresses. The base-token equality check does not validate pool identity or version.

N-10 · Migration sends the burn amount to the zero address

Type Informational · **Assumption** The transfer represents the intended burn mechanism

Fact. The migrator transfers `amountToBurn` to `address(0)`. `ADXTOKEN` permits that transfer. The transfer does not reduce `totalSupply`.

`ADXSupplyController` counts burns held at a different address when it calculates minting headroom.

Preconditions. A migration must occur after the grace period and calculate a positive burn amount.

Consequence. The transferred ADX becomes unspendable. Its value remains in `ADXTOKEN.totalSupply`. It does not increase the controller's recorded burn balance.

Recommendation. Confirm that the supply cap should treat these tokens as outstanding. Use the controller's recognized burn mechanism when migration should increase minting headroom.

Verification. The reviewed migrator sends the burn amount to the zero address. The controller tracks a different burn address.

N-11 · Balance views can include incentives that the controller cannot mint

Type Informational

Fact. `StakingPool.totalSupply` adds the full value returned by `mintableIncentive`.

`SupplyController.innerMint` rejects minting above its cap. `mintableIncentive` does not apply the same cap.

Preconditions. Accrued incentives must exceed the remaining minting headroom.

Consequence. `totalSupply`, `shareValue`, and `balanceOf` can report value that is not present in the pool.

A normal exit can also revert when it first calls `mintIncentive`. The `skipMint` exit path calculates payment from the pool's actual ADX balance.

Recommendation. Limit the pending incentive included in pool views to the controller's available headroom. Expose any excess accrual separately.

Verification. The pool and controller use different calculations. The reviewed tests do not cover the supply-cap boundary.

N-14 · Governance can set the rage-leave payout to zero

Type Informational · **Assumption** Governance is trusted

Fact. `setRageReceived` enforces an upper bound of 1000 promille. The function has no lower bound.

The source still contains the comment `// AUDIT: should there be a minimum here?`.

Preconditions. Governance must set `rageReceivedPromilles` to zero or another unexpectedly low value.

Consequence. A user who rage-leaves can receive little or no ADX while the function burns their shares.

Recommendation. Set a minimum payout in code, or document that governance may set the value to zero. Remove the unresolved audit comment.

Verification. The reviewed function has no lower bound. The source still contains the audit comment.

N-18 · External self-calls add gas to common paths

Type Informational

Fact. The pool calls `this.shareValue()` and `this.totalSupply()` from its own functions. These expressions create external calls to the same contract.

`balanceOf` and both transfer functions use these paths.

Preconditions. A user or integration must call a function that reads the share value or total supply.

Consequence. The external call boundary adds gas without changing the result.

Recommendation. Move each calculation into an internal helper. Keep external wrappers only for the public interface.

Verification. The reviewed source contains external self-calls at `StakingPool.sol:27,37,48,156`.

Appendix: verification and limitations

goodmorning verified N-04 by compiling the staking sources through `solc-js` Standard JSON. Each compiler version ran separately.

COMPILER	STAKING CONTRACTS
0.7.5 and 0.7.6	Rejected because required <code>override</code> markers were missing
0.8.7, pinned in <code>truffle-config.js</code>	Rejected with 13 missing <code>override</code> errors
0.8.8	Accepted

The repository command `npm run truffle compile` was unavailable during review because the dependency tree was not installed.

goodmorning checked N-07 against the deployed v1 pool source. That source shows that v1 transfers can move staged unbonding shares.

The review used static source inspection for every other finding. The current tests do not cover the reviewed v2 pool and migrator paths.

`test/TestStakingPool.js` targets functions removed in commit `43a352f`. `StakingPoolMigratorGovernance` has no tests in the reviewed branch.

In this report, Resolved records the stated correction in commit `a7c82a9`. Runtime regression coverage remains absent unless this report states otherwise.

Add regression tests for every resolved and open transaction path before deployment.